

Capítulo 6

Representações Numéricas

Vimos como representar quantidades em binário assim como em outras bases posicionais tal como a base octal e hexadecimal. Até o momento estudamos apenas como representar números naturais $\mathbb{N}$. Para representar números em outros conjuntos numéricos como por exemplo números inteiros $\mathbb{Z}$ ou reais $\mathbb{R}$ adaptações se fazem necessárias.

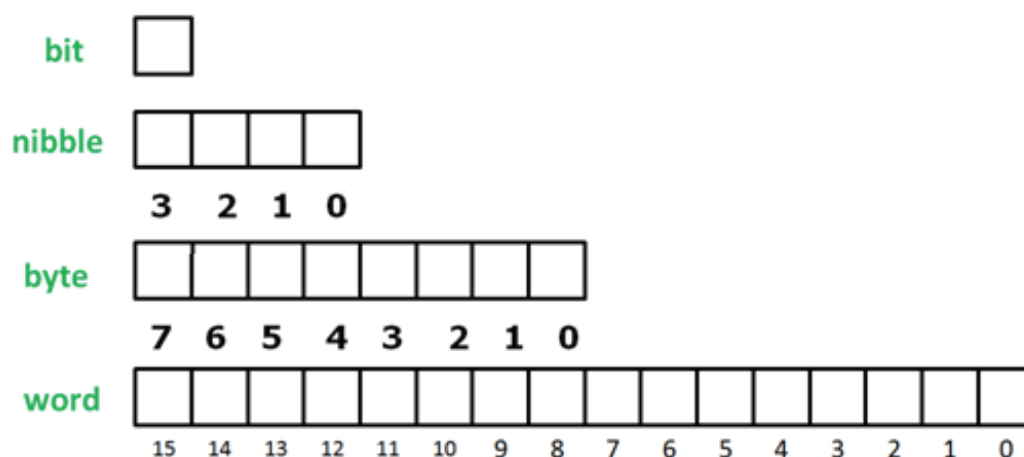


Figura 6.1: Nomes especiais dados a conjuntos de bits. Um bit unitário, 4 bits ou nibble, 8 bits ou byte e um número pré-especificado e usualmente múltiplo de 8 chamado word.

A primeira delas refere-se a necessidade de se definir a priori quantos bits serão utilizados para representar números, seja qual for a conjunto numérico em questão. Vale lembrar que em matemática lidamos com conceitos abstratos, e o sistema posicional nos permite quantos símbolos necessitarmos (possivelmente infinitos). Em sistemas digitais o mesmo não se aplica.

Como um sistema digital deve ser construído fisicamente, não importa quão poderoso for o sistema ele será invariavelmente finito. isto significa que limites devem ser impostos. Um dos limites mais importantes refere-se ao número de bits que serão suportados pelo sistema. A figura 6.1 resume os principais agrupamentos notáveis de bits. Um byte refere-se a um conjunto de 8 bits. Sendo assim, sistemas que utilizam 8 bits, podem processar números que variam na faixa $[0, 255]$ ou seja $[00000000, 11111111]$. Como se pode notar um byte nos permite representar apenas quantidades restritas. A adoção de um número maior de bits é então desejável pois permite a representação de quantidades maiores.

6.1 Números Inteiros ($\mathbb{Z}$) em Binário

O conjunto dos números inteiros (6.1) é composto por infinitos elementos. Uma representação discreta dos números inteiros é naturalmente impossível pois por definição qualquer representação discreta será limitada pelo número de bits disponíveis. O primeiro importante fato acerca da representação de números inteiros em sistemas computacionais é definir seu **espaço representacional**. Essencialmente, refere-se ao número de bits disponíveis para a representação numérica.

$$\mathbb{Z} = \{-\infty, \dots, -2, -1, 0, 1, 2, \dots, \infty\} \quad (6.1)$$

Adicionalmente, para representar quantidades inteiras (que incluem números negativos) faz-se necessário que sejamos capazes de indicar se a quantidade representada é positiva ou negativa. Este não é um problema em matemática, pois podemos simplesmente utilizar o sinal “-” para sinalizar um número negativo. Em um sistema digital não dispomos desta possibilidade. A representação do sinal deve existir fisicamente no escopo do sistema digital.

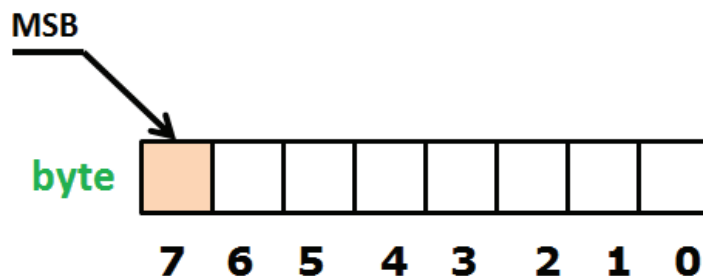


Figura 6.2:

6.1.1 Bit de Sinal

Uma solução simples seria "reservar" um bit, geralmente o mais significativo para indicar o sinal (+ ou negativo). Esta é uma solução conveniente, pois como se faz necessário representar apenas dois sinais e um bit pode ser 0 ou 1. Faz-se necessário convencionar que valor será utilizado para o sinal + e complementarmente $-$. Por convenção, utilizamos 0 para + e 1 para $-$.

A figura 6.3 apresenta um exemplo de um número inteiro representado utilizando-se a técnica de bit de sinal. Como o bit de sinal é "0" conclui-se que o número em questão é positivo.

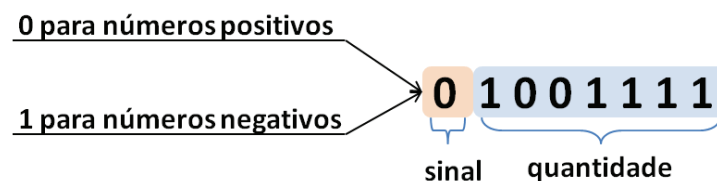


Figura 6.3:

A seguir, são apresentados alguns exemplos de números inteiros utilizando o bit mais significativo para marcar o sinal:

Exemplo 6.1. Converta os seguintes números inteiros que utilizam o bit de sinal para decimal:

- $1\ 0\ 0\ 0\ 0\ 0\ 1 \rightarrow -1_{10}$
- $1\ 0\ 0\ 0\ 1\ 0\ 1\ 0 \rightarrow -10_{10}$
- $0\ 0\ 0\ 0\ 1\ 0\ 0\ 1 \rightarrow 9_{10}$
- $1\ 0\ 1\ 0\ 1\ 0\ 1\ 0 \rightarrow -42_{10}$
- $0\ 0\ 0\ 0\ 0\ 0\ 0\ 0 \rightarrow 0_{10}$
- $1\ 0\ 0\ 0\ 0\ 0\ 0\ 0 \rightarrow -0_{10}?$

O exemplo anterior apresenta um grande problema com a representação de bit de sinal. Como se pode ver nos últimos dois números, temos 0 e -0 o que naturalmente não faz sentido pois o número zero não é nem positivo nem negativo.

Por esse motivo buscaram-se outras formas de codificação de números inteiros. Vale ressaltar que a necessidade de se codificar o sinal diretamente na palavra de dados é uma constante nas formas de representação em voga.

6.1.2 Complemento de 1

A primeira possibilidade é utilizar o que ficou convencionado chamar de complemento de 1. O nome é confuso, pois o método limita-se a simplesmente

complementar, ou seja, inverter todos os bits para representar a quantidade negativa desejada. O esquema de codificação dita que um bit da palavra disponível deve ser reservado, geralmente o mais significativo, e não pode ser utilizado para representação de quantidades. Os números positivos são representados incrementalmente tal como nos números naturais até o limite de $N - 1$ bits, onde N é o comprimento em bits da palavra. Para obter-se a representação negativa de qualquer quantidade basta que sua representação positiva seja invertida. A tabela 6.1 apresenta o código de complemento de 1 para palavras de 4 bits.

Tabela 6.1: Correspondência entre a quantidade em decimal e sua representação codificada em complemento de 1.

Decimal	Complemento de 1
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1110
-2	1101
-3	1100
-4	1011
-5	1010
-6	1001
-7	1000
-0	1111

Tal como ocorre no método de bit de sinal, em complemento de 1 há duas representações para a quantidade zero (0000 e 1111).

Exemplo 6.2. Considere os exemplos de representação em complemento de 1 e sua correspondência com a representação decimal:

$$\begin{array}{ll}
 00000001_{C1} \longrightarrow 1_{10} & 11111110_{C1} \longrightarrow -1_{10} \\
 00001010_{C1} \longrightarrow 10_{10} & 11110101_{C1} \longrightarrow -10_{10} \\
 00001001_{C1} \longrightarrow 9_{10} & 11110110_{C1} \longrightarrow -9_{10} \\
 00101010_{C1} \longrightarrow 42_{10} & 11010101_{C1} \longrightarrow -42_{10} \\
 00000000_{C1} \longrightarrow 0_{10} & 11111111_{C1} \longrightarrow -0_{10}
 \end{array}$$

6.1.3 Complemento de 2

Para solucionar o problema de duas representações para a quantidade zero foi proposto o esquema de complemento de 2. O esquema dita que o bit mais significativo da palavra deve ser reservado, ou seja, não pode ser utilizado para representação de quantidades, sejam elas negativas ou positivas. A seguir o esquema especifica que as quantidades positivas são representados incrementalmente tal como nos números naturais até o limite de $N - 1$ bits, onde N é o comprimento em bits da palavra. Para obter-se a representação negativa de qualquer quantidade basta que sua representação positiva seja invertida e subsequentemente somada a 1. A tabela 6.2 apresenta o código de complemento de 2 para palavras de 4 bits.

Tabela 6.2: Correspondência entre a quantidade em decimal e sua representação codificada em complemento de 2.

Decimal	Complemento de 1
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

Note que este código admite apenas uma representação para a quantidade zero. Alguns exemplos são apresentados a seguir utilizando palavras de 16 bits :

$$\begin{array}{llll}
 1_{10} \longrightarrow & & & 00000000 \ 00000001_{C2} \\
 -10_{10} \longrightarrow & 11111111 \ 11110101_{C1} \longrightarrow & & 11111111 \ 11110110_{C2} \\
 -9_{10} \longrightarrow & 11111111 \ 11110110_{C1} \longrightarrow & & 11111111 \ 11110111_{C2} \\
 -42_{10} \longrightarrow & 11111111 \ 11010101_{C1} \longrightarrow & & 11111111 \ 11010110_{C2} \\
 0_{10} \longrightarrow & & & 00000000 \ 00000000_{C2}
 \end{array}$$

O método de complemento de 2 é o adotado efetivamente na implementação de circuitos aritméticos em $\mathbb{Z}$ por motivos que ficarão óbvios no capítulo 9. A seguir trataremos de esquemas de codificação de números em $\mathbb{R}$.

6.2 Números Reais ($\mathbb{R}$) em Binário

O conjunto dos números reais (6.1) é composto por infinitos elementos. Adicionalmente, há infinitos números reais no intervalo definido por quaisquer par de números reais distintos. Uma representação discreta dos números do conjunto de números reais em sua plenitude é naturalmente impossível pois por definição qualquer representação discreta será limitada pelo número de bits disponíveis.

O primeiro importante fato acerca da representação de números reais em sistemas computacionais é definir seu **espaço representacional**. Essencialmente, refere-se ao número de bits disponíveis para a representação numérica. O segundo fato importante refere-se a como codificar as partes inteira e a fracionária da quantidade em questão. Antes de apresentarmos duas formas de codificação possíveis vale revisar como se processa o mecanismo de conversão de quantidades de real decimal para binário e vice-versa.

6.2.1 Conversão de Real Decimal para Binário

Livre da restrição do tamanho de palavras a representação real em binário é apresentada na figura 6.4. Como se pode notar a potência associada as casas inteiras segue o mesmo modelo apresentado para o conjunto dos números inteiros (5.7). A novidade refere-se a potência associada as casas fracionárias. Em via de regra, da esquerda para a direita, ou seja, da casa mais para a menos significativa, associa-se as casas fracionárias potências negativas. Sendo assim, a casa imediatamente a direita da vírgula equivale a $2^{-1} = 0.5$, a segunda casa mais significativa $2^{-2} = 0.25$ e assim sucessivamente, tal como apresentado na tabela 6.3.

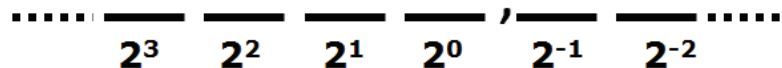


Figura 6.4: Potências associadas as seis primeiras 6 casas da representação real em binário.

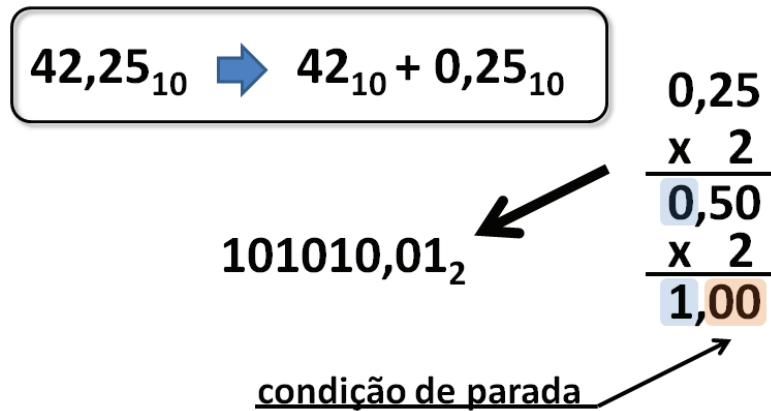
Tabela 6.3: Potências associadas as dez casas mais significativa a direita da vírgula.

Potência	Valor
2^{-1}	0,5
2^{-2}	0,25
2^{-3}	0,125
2^{-4}	0,0625
2^{-5}	0,03125
2^{-6}	0,015625
2^{-7}	0,0078125
2^{-8}	0,00390625
2^{-9}	0,001953125
2^{-10}	0,0009765625

O processo de conversão de números reais de decimal para binário segue os seguintes passos:

1. Separar a parte inteira da fracionária;
2. converter a parte inteira utilizando uma das técnicas apresentadas no capítulo 5;
3. multiplicar a parte fracionária por 2;
4. atribua a parte inteira do resultado da multiplicação a próxima casa a direita da vírgula ainda não preenchida;
5. caso a parte inteira do resultado da multiplicação seja 1 deve-se subtrair um do resultado total da multiplicação;
6. repita o passo anterior até que o resultado da multiplicação seja zero ou que o número disponível de casas disponíveis tenha sido alcançado.

Exemplo 6.3. Tome como exemplo o número $42,25_{10}$. Inicialmente separam-se a parte inteira da fracionária, ou seja, $pi = 42_{10}$ e $pf = 0,25_{10}$. A seguir procede-se com a conversão da parte inteira resultando em $pi = 101010_2$. A seguir multiplicamos a parte fracionária por 2 obtendo $pf_{-1} = 0,2 \times 2 = 0,4_{10}$. Atribui-se então a parte inteira do resultado da multiplicação a primeira casa a direita da vírgula. A próxima multiplicação resulta em $0,4 \times 2 = 0,8$. Atribui-se "1" a segunda casa a direita da vírgula e como a parte inteira da multiplicação resulta em número inteiro, deve-se subtrair um da multiplicação, resultando em 0,8 e conseqüentemente o final do processo de conversão.



O processo de conversão infelizmente pode gerar situações nas quais um número que na base 10 possua representação exata gere uma dízima periódica na base 2. Considere por exemplo o número $15,2_{10}$. Sua conversão para a base dois resulta em $1111,001001001 \dots$.

6.2.2 Representação de Ponto Fixo

O primeiro esquema de representação binária para números reais chamado de representação de ponto fixo subdivide o número de bits disponíveis N entre bits a serem usados para representar a parte fixa N_I e o bits para a parte fracionária N_F , onde $N = N_I + N_F$. Essencialmente o que se faz é fixar a vírgula na transição entre dois bits da palavra. Um exemplo pode ser visualizado na figura 6.5.

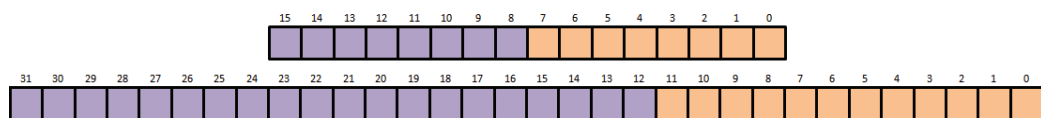


Figura 6.5: Exemplos de palavras que codificas para números reais em ponto fixo. Acima: parte inteira em roxo (bits 15-8) e parte fracionária em laranja (7-0) e Abaixo: parte inteira em roxo (bits 31-12) e parte fracionária em laranja (11-0).

Nos exemplos apresentados na figura 6.5 tem-se um fator de escalonamento $FE = 8$ e mantissa de 16 bits e $FE = 12$ e uma mantissa de 32 bits, respectivamente. Números reais podem ser naturalmente positivos ou negativos. Consequentemente deve-se adotar um esquema para diferenciação entre números positivos e negativos. Por conversão adota-se o esquema de

complemento de 2.

Exemplo 6.4. Considere o número 42,5. Sua representação em notação de ponto fixo utilizando o complemento de 2 como esquema de sinalização começa com a conversão do número para binário e uma mantissa de 16 bits e $FE = 8$. A seguir aloca-se a fracionária da esquerda para a direita para as casas reservadas pelo fator de escalonamento. Essencialmente o fator de escalonamento dita quantas casas depois da vírgula serão representadas. Caso a fração seja mais longa que o fator de escalonamento ela será truncada.

$$\begin{aligned} 42,5 &\longrightarrow 101010,1 \\ 00101010\ 10000000 &\quad (\text{ponto fixo}) \end{aligned}$$

Exemplo 6.5. Considere o número $-14,25$ a ser representado utilizando a notação de ponto fixo com mantissa de 16 bits e $FE = 4$.

$$\begin{aligned} 14,25 &\longrightarrow 1110,01 \\ 000000001110\ 0100 &\quad 14,25 \text{ (ponto fixo)} \\ -14,25 &\longrightarrow ?_{C2} \\ 111111110001\ 1011 &\quad -14,25 \text{ (compl. de 1)} \\ 111111110001\ 1100 &\quad -14,25 \text{ (compl. de 2/ponto fixo)} \end{aligned}$$

Naturalmente ao somar $14,25_{C2}$ com $-14,25_{C2}$ ou seja, $000000001110\ 0100$ com $111111110001\ 1100$ obtêm-se $000000000000\ 0000$.

Limitações do Espaço Representacional

Embora o conjunto Real seja infinito sua representação computacional discreta impõe claramente um limite ao número de quantidades Reais distintas que podem ser representadas. Mais especificamente, o que ocorre caso se deseje representar um número estritamente inteiro (ex. 255), ou um número estritamente fracionário (ex. 0,00390625)? Considere para fins do desenvolvimento do raciocínio palavras com mantissa de 16 bits e $FE = 8$. Claramente no primeiro caso, todos os bits reservados para a parte fracionária (8 bits menos significativos) serão desperdiçados. O mesmo ocorre com a representação de números estritamente fracionários, porém neste caso, os 8 bits mais significativos é que serão desperdiçados. Em suma, a representação de ponto fixo não é utilizada na prática pois ela utiliza mal o espaço representacional.

Erros Representacionais

O conceito de erros de representação refere-se a incapacidade devido as limitações impostas pelo espaço representacional de se representar certas quantidades.

Exemplo 6.6. Tome como exemplo o número $10,2$ a ser representado utilizando a notação de ponto fixo com uma mantissa de 16 bits e $FE = 6$. A conversão de decimal para binário resulta em uma dízima periódica muito embora a representação decimal seja exata.

$$10,25_{10} \cong 1010,0011001100110011 \dots$$

como o espaço representacional adotado é de 16 bits com $FE = 6$ faz-se necessário truncar a dízima para 6 casas resultando na representação em ponto fixo

$$0000001010\ 001100$$

Procedendo com a conversão da representação em ponto fixo para decimal obtêm-se $2^3 + 2^1, 2^{-3} + 2^{-4} = 10,1875_{10}$.

O erro imposto pela necessidade de se codificar o número em um número finito de bits pode ser calculado pela diferença entre o valor exato decrescido do valor representado resultado em $\Delta = \text{exato} - \text{representado} = 10,25 - 10,1875 = 0,0625$.

Quanto maior o número de bits reservados para a parte fracionária menor será o erro representacional. Infelizmente o número de bits reservados para a parte inteira diminuirá na mesma proporção e conseqüentemente quantidades inteiras menores serão representáveis. Esta é a principal motivação para a adoção de um sistema representacional distinto para números reais.

6.2.3 Representação de Ponto Flutuante

Outra forma para a representação de números reais em computador refere-se a utilização da notação de ponto flutuante. A ideia básica refere-se a codificar nos bits disponíveis de uma palavra utilizando uma variante da notação científica. O exemplo 6.7 ilustra a mecânica que permeia a notação científica. A ideia é utilizar a operação de potenciação para isolar na parte inteira da representação numérica apenas um dígito.

Exemplo 6.7. $42,42 = 42,42 \times 10^0 = 4,242 \times 10^1$

A notação de ponto flutuante segue as mesmas linhas gerais. No entanto a base da potenciação utilizada é a 2 e durante o processo de potencialização desloca-se para a parte inteira o primeiro um da quantidade. Isso se dá porque como a parte inteira sempre será sempre "um", este não precisará ser

representado. Consequentemente a parte inteira do número não precisa ser codificada nos bits disponíveis.

$$Q_2 = [\pm]0, mantissa \times 2^{expoente+exc} \quad (6.2)$$

O *expoente* será o número de vezes que a vírgula foi deslocado. Caso o deslocamento seja para a esquerda o número será positivo ou negativo caso o deslocamento seja para a direita. O *expoente* será adicionado ao excesso *exc* que dependerá do número de bits reservados para o expoente na razão de $2^{|expoente|-1}$.

Exemplo 6.8. Considere por exemplo que desejamos representar o número 42,42 utilizando a notação de ponto flutuante utilizando uma palavra de 32 bits com 23 bits de mantissa e 8 bits para o expoente. Primeiramente converte-se o número em decimal para binário obtendo-se a representação 101010,0110101110000101000111... A seguir desloca-se a vírgula para antes do primeiro 1 da representação obtendo-se 1,010100110101110000101000111... O próximo passo refere-se a separar os bits representáveis na mantissa. Como a mantissa possui 23 bits, recuperamos o mesmo número de bits após a vírgula para serem transpostos para a mantissa resultando 01010011010111000010100. Como o número é positivo o bit mais significativo referente ao sinal será 0 e por fim precisamos calcular o expoente. A vírgula foi deslocada 5 casas para a esquerda, logo o expoente desejado será +5. Para 8 bits de expoente utilizamos o código de excesso de 127, ou seja, somamos o expoente a 127 resultando em $127+5 = 132 \equiv 10000100_2$. O resultado final da representação de 42,42 em notação de ponto flutuante será

$$0 \ 10000100 \ 01010011010111000010100_F$$

O Padrão IEEE 754

IEEE 754 [2] é um padrão técnico para representação de número em ponto flutuante introduzido em 1985 pelo *Institute of Electrical and Electronics Engineers* (IEEE). Além de especificar as regras de codificação o padrão ainda define casos especiais, e nomeia diferentes espaços representacionais, notavelmente a representação de precisão simples que utiliza 32 bits e a de precisão dupla para 64 bits.

Precisão Simples

A precisão simples é o primeiro espaço representacional de interesse. Ela permite que um número real seja representado utilizando palavras de 32 bits. Como se pode observar na figura 6.6 o bit mais significativo (bit 31) é

para a posição apropriada dada pelo expoente 11, 0010010000111111011011. Procede-se então com o método de conversão de binário para decimal visto anteriormente obtendo-se o número 3, 1415927410125732421875.

Em verdade está é a representação do número $\pi = 3,1415926535$ no qual apenas as dez primeiras casas iniciais são consideradas. Note que há uma discrepância entre o número real e o representado resultado em um erro $\Delta \approx 8,7512 \times 10^{-8}$. Embora o erro seja pequeno, ele demonstra diversos fatos interessantes. O primeiro deles refere a impossibilidade de se representar exatamente muitos números. O segundo fato é indireto porém de extrema importância. Considere o caso em que cálculos sucessivos são executados utilizando representações imprecisas de números reais por meio da codificação em ponto flutuante. Claramente os erros se acumularão e ao final de centenas ou milhares de cálculos o erro acumulado pode vir a não ser tão insignificante quanto uma diferença na oitava casa decimal.

Tal problema pode ser mitigado, porém não eliminado, utilizando palavras maiores, expandindo assim o conjunto e precisão do espaço representacional.

Precisão Dupla

A precisão dupla utiliza o dobro de bits em comparação a precisão simples, ou seja, 64 bits. A mecânica de codificação e decodificação é idêntica a vista no caso anterior respeitada as diferenças introduzidas por mais bits reservados para a mantissa e expoente. A figura 6.7 apresenta um modelo esquemático da subdivisão de bits adotada pela precisão dupla.



Figura 6.7: Subdivisão da palavra de 64 bits em campos previstos pelo espaço representacional de precisão dupla.

$$Q_2 = (-1)^s \times \left(1 + \sum_{i=1}^{52} b_{52_i} \times 2^i \right) \times 2^{e-1023} \quad (6.4)$$

Casos Especiais

Existem alguns valores especiais que são representados de uma forma padrão nessa norma IEEE 754, visto que esses valores especiais tem a característica de a parte reservada para o expoente (E) é toda preenchida com 0's ou toda preenchida com 1's e a parte reservada para a mantissa (M) é

IEEE 754 - Precisão Simples			Valor	
s	e	m		
0	0000 0000	000 0000 0000 0000 0000 0000	+0	Zero
1	0000 0000	000 0000 0000 0000 0000 0000	-0	
0	1111 1111	000 0000 0000 0000 0000 0000	+inf	Infinito Positivo
1	1111 1111	000 0000 0000 0000 0000 0000	-inf	Infinito Negativo
0	1111 1111	010 0000 0000 0000 0000 0000	+NaN	Not a Number
1	1111 1111	010 0000 0000 0000 0000 0000	-NaN	

Tabela 6.4: Tabela de valores especiais em ponto flutuante.

preenchida com somente 0's, a não ser que o número não seja normalizado, veja exemplos dessa representação na tabela a seguir:

Exercícios

1. Considerando números naturais, qual a faixa de valores que podem ser representados utilizando-se 8, 10, 16, 32, 64 e 128 bits?
2. Represente os seguintes números em binário utilizando a notação de inteiros sinalizados. Utilize palavras de 16 bits:

(a) -42_{10}

(b) -808_{10}

(c) -78_{10}

(d) -4200_{10}

(e) -32766_{10}

(f) -131_{10}

(g) -16383_{10}

(h) -5429_{10}

3. Converta os seguintes números para decimal. Considere o bit mais significativo como sendo bit de sinal:

(a) 00101010_2

(b) 11111111_2

(c) 10101010_2

(d) 00000000_2

(e) 00001111_2

(f) 11110000_2

(g) 01010101_2

(h) 00001111_2

4. Escreva os seguintes números sob a forma de somas de suas potências:

(a) $42,42_{10} = 4x10^1 + 2x10^0 + 4x10^{-1} + 2x10^{-2}$

(b) $1024,1010_{10} =$

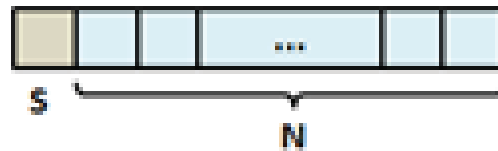
(c) $40,96_{10} =$

(d) $81,91_{10} =$

(e) $1010,10_2 =$

(f) $111111,11110_2 =$

- (g) $1001, 1001_2 =$
 - (h) $011110, 1010_2 =$
 - (i) $77, 7_8 =$
 - (j) $42, 24_8 =$
 - (k) $F0F, A_{16} =$
 - (l) $BA, BACA_{16} =$
 - (m) $B1, BA_{16} =$
 - (n) $DA, D05_{16} =$
 - (o) $BEB1, DA_{16} =$
5. Quantos bits equivalem a:
- (a) 1 byte
 - (b) 1 Pbyte
 - (c) 1 Kbyte
 - (d) 1 Ebyte
 - (e) 1 Mbyte
 - (f) 1Zbyte
 - (g) 1 Gbyte
 - (h) 1Ybyte
 - (i) 1Tbyte
6. É comum nomear o primeiro e o último bit de uma palavra. O que significa MSB e LSB neste contexto?
7. Considere a seguinte representação de números inteiros sinalizados:



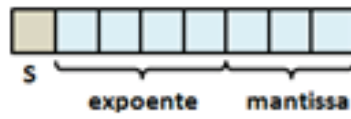
Quais são o maior e menor inteiro representável por palavras com o seguinte número de bits:

- (a) 2 bits
- (b) 16 bits

- (c) 4 bits
 - (d) 32 bits
 - (e) 8 bits
 - (f) 64 bits
8. Escreva todos os números (positivos e negativos) representáveis em complemento de 1 para as palavras dos seguintes tamanhos:
- (a) 3 bits
 - (b) 5 bits
 - (c) 4 bits
 - (d) 6 bits
9. A existência de dois zeros na representação em complemento de 1 traz alguma desvantagem? Justifique.
10. Escreva todos os números (positivos e negativos) representáveis em complemento de 2 para as palavras dos seguintes tamanhos:
- (a) 3 bits
 - (b) 5 bits
 - (c) 4 bits
 - (d) 6 bits
11. Dados os complementos de 2 abaixo, verifique a que decimal negativo eles pertencem.
- (a) -1011101
 - (b) -1010101100
 - (c) -11010111
 - (d) -1011110111
 - (e) -101110111
 - (f) -11011011
12. Quanto vale em decimal os seguintes números representados em ponto flutuante. Considere 1 bit para sinal 4 para expoente e 3 para mantissa:
- (a) 00000001
 - (b) 01000001

- (c) 10000111
- (d) 10111010
- (e) 00010101
- (f) 11011011

13. Considere a seguinte representação em ponto flutuante:



Quais são o maior e menor números representados por esta palavra?